IMAGE FORGERY DETECTION

Using Deep Learning

Project Report

By
Deeksha Reddy Patlolla - 111444513
Spandana Erukulla - 111447474

Repository	https://github.com/Deekshareddy10/Image_forgery_detection_using_DL/tree/main
Technology	Python 3.10 · TensorFlow · Flask · OpenCV
Model	U-Net with DCT Preprocessing
Dataset	CoMoFoD (Copy-Move Forgery Detection)

1. Problem Statement

Digital image manipulation has become alarmingly accessible. Tools such as Adobe Photoshop, GIMP, and AI-based inpainting software allow virtually anyone to create convincing forgeries in minutes. The two most prevalent techniques are:

- Copy-Move Forgery: A region within an image is copied and pasted to another location in the same image - typically to conceal or duplicate objects.
- Splicing Forgery: Regions from different images are combined to create a composite that misrepresents reality.

These forgeries pose serious risks across multiple domains: news media fabricates events, legal evidence is tampered with, scientific publications use manipulated figures, and social media spreads disinformation. The human visual system is ill-equipped to detect subtle manipulations, making automated detection essential.

This project addresses the problem of automatically detecting and localizing forged regions in digital images using a deep learning pipeline, outputting a pixel-level mask that highlights tampered areas and classifying the image as REAL or FORGED.

2. Motivation

2.1 Application Importance

The proliferation of fake media has created an urgent demand for reliable automated forgery detection systems. Key application domains include:

- Journalism & Fact-Checking: News organizations need tools to verify image authenticity before publishing.
- Law Enforcement & Legal Systems: Digital images submitted as evidence must be verified for integrity.
- Academic Integrity: Research papers increasingly rely on images that may be manipulated.
- Insurance Claims: Fraudulent photo evidence used to inflate claims can be detected automatically.
- Social Media Platforms: Automated pipelines can flag suspicious images at scale.

2.2 Gap in the ML Community

While many traditional methods rely on handcrafted features (e.g., SIFT, block-matching, DCT statistics alone), they often struggle with post-processing such as JPEG compression, noise addition, or geometric transformations applied to hide the manipulation. The combination of frequency-domain preprocessing (DCT) with an end-to-end trainable segmentation network (U-Net) addresses this gap by learning discriminative features that are both spatially aware and frequency-sensitive.

2.3 Ethical Considerations

This project raises several ethical matters that must be acknowledged:

- Dual-Use Risk: A forgery detector could be reverse-engineered to craft adversarial manipulations that evade detection. Responsible disclosure and access control are important.
- Fairness & Bias: The model is trained on CoMoFoD, which may not represent the full diversity of image sources (cameras, demographics, cultural contexts). Biased training data could lead to unequal detection rates across different image types.
- Over-Reliance: Automated tools should augment human judgment, not replace it. False positives could unjustly flag authentic images; false negatives could allow forgeries to pass undetected.
- Privacy: The system processes user-uploaded images. Care must be taken to avoid storing or misusing submitted content.

The project explicitly includes a disclaimer in its documentation that it is a proof-of-concept and not intended for forensic-grade professional use.

3. Related Works

Image forgery detection has been studied extensively. Below are the most relevant prior works:

3.1 Traditional / Handcrafted Feature Methods

- SIFT-based Copy-Move Detection (Lowe, 2004): Uses Scale-Invariant Feature Transform to find matching keypoints between image regions. Effective but brittle under noise and compression.
- DCT Block Analysis (Popescu & Farid, 2004): Divides the image into overlapping blocks, computes DCT coefficients, and searches for duplicate blocks. Foundational to our preprocessing step.

- Passive Detection using DCT and LBP (Alahmadi et al., 2017): Combines DCT statistics with Local Binary Patterns for richer feature representation.

3.2 Deep Learning Methods

- ManTraNet (Wu et al., CVPR 2019): A fully convolutional network that detects and localizes image manipulations by computing local anomaly maps. Generalizes across forgery types but is complex to train.
- CAT-Net (Kwon et al., CVPR 2021): Uses JPEG compression artifacts traced via DCT streams, combined with a segmentation head, to localize forgeries. Directly inspired the DCT-preprocessing motivation of this project.
- SPAN — Spatial Pyramid Attention Network (Hu et al., 2020): Employs multi-scale attention to capture both local and global forgery cues.
- U-Net for Medical Image Segmentation (Ronneberger et al., 2015): Originally designed for biomedical imaging, the U-Net encoder-decoder architecture with skip connections is highly effective for pixel-level segmentation tasks — including forgery mask prediction.

3.3 Datasets

- CoMoFoD (Tralic et al., 2013): 260 image sets covering copy-move forgeries with various post-processing transformations (rotation, scaling, noise, compression). Used in this project. Dataset link: https://www.vcl.fer.hr/comofod/
- CASIA v1.0 / v2.0: Widely used benchmark for both copy-move and splicing detection.
- Columbia Uncompressed Image Splicing Dataset: Uncompressed images ideal for splicing detection evaluation.

4. Method

4.1 System Architecture Overview

The system follows a two-stage pipeline:

1. Preprocessing: Convert the input image to the frequency domain using the 2D Discrete Cosine Transform (DCT) applied to non-overlapping 8×8 blocks.
2. Segmentation: Feed the DCT-transformed representation into a U-Net model that predicts a binary forgery mask at pixel resolution.

4.2 DCT Preprocessing

The Discrete Cosine Transform decomposes image blocks into frequency components. Copy-move and splicing operations disrupt the natural frequency statistics of an image region. By operating in the frequency domain, the model can exploit these disruptions as discriminative signals, complementing spatial RGB features. Each 8×8 block's DCT coefficients are used as input channels, making the model sensitive to compression artifacts and frequency anomalies introduced by manipulation.

4.3 U-Net Architecture

The U-Net is a fully convolutional encoder-decoder network characterized by:

- Encoder path: Successive convolutional blocks with max-pooling to capture multi-scale features.
- Bottleneck: The deepest feature representation capturing global context.
- Decoder path: Upsampling operations that progressively restore spatial resolution.
- Skip connections: Direct connections from encoder to corresponding decoder layers, preserving fine spatial details critical for accurate mask boundary localization.

The final output is a single-channel probability map (sigmoid activation) representing the likelihood of forgery at each pixel. A threshold converts this to a binary mask.

4.4 Classification

After generating the forgery mask, the percentage of pixels classified as forged is computed. If this percentage exceeds a configurable threshold (empirically tuned), the image is classified as FORGED; otherwise, it is classified as REAL. This dual output - mask + label - provides both localization and classification.

4.5 Web Application (Flask)

A Flask backend serves the model inference. Users upload images via a Bootstrap-based UI; the backend loads the Keras model, runs the DCT preprocessing and U-Net inference, and returns the predicted mask overlaid on the original image alongside the REAL/FORGED verdict.

5. Reproduction Instructions

5.1 Requirements

Python Version	3.10 or higher
Key Libraries	TensorFlow 2.x, Flask 3.0, OpenCV-Python, NumPy, Gunicorn

Version Control	Git + Git LFS (for model file)
Repository	https://github.com/Deekshareddy10/Image_forgery_detection_using_DL/tree/main

5.2 Installation Steps

3. Clone the repository: `git clone https://github.com/Shubham-711/image-forgery-detector.git`
4. Create and activate a virtual environment: `python -m venv venv && source venv/bin/activate`
5. Install dependencies: `pip install -r requirements.txt`
6. Pull the model file via Git LFS: `git lfs pull`
7. Set a secure secret key in app.py (use: `python -c "import secrets; print(secrets.token_hex(24))"`)
8. Launch the app: `flask run` -> Navigate to `http://127.0.0.1:5000`

5.3 requirements.txt (representative)

Package	Version / Notes
`tensorflow`	2.x (Keras API)
`flask`	3.0
`opencv-python`	latest stable
`numpy`	latest stable
`gunicorn`	for production deployment
`Pillow`	image handling

6. Limitations

The following weaknesses have been identified in the current implementation:

- Dataset Bias: The model is trained exclusively on CoMoFoD. Performance may degrade significantly on images from domains not represented in the training set (e.g., medical images, satellite imagery, smartphone snapshots).

- Forgery Type Coverage: The system is designed for copy-move and splicing forgeries. It does not detect inpainting, morphing, GAN-generated deepfakes, or subtle lighting/color adjustments.
- JPEG Compression Sensitivity: Aggressive JPEG compression of the input image degrades DCT coefficient quality, potentially reducing detection accuracy.
- Threshold Sensitivity: The pixel-percentage threshold for REAL/FORGED classification is empirically chosen and may require tuning for different use cases.
- No Adversarial Robustness: The model has not been hardened against adversarial attacks — a determined adversary could craft perturbations that cause the model to misclassify forged images as real.
- Proof-of-Concept Scale: The Flask server is single-threaded by default and not designed for high-throughput production use without further optimization (e.g., batching, async inference, GPU deployment).
- Generalization to Modern AI Manipulations: AI-generated deepfakes and diffusion model inpainting represent a new class of forgery that this architecture was not trained to handle.

7. Conclusion

This project demonstrates a functional end-to-end pipeline for detecting and localizing copy-move and splicing forgeries in digital images. By combining frequency-domain preprocessing (DCT) with the spatially-aware U-Net architecture, the system achieves pixel-level forgery mask prediction and binary image classification through a clean web interface.

The project contributes a reproducible, deployable proof-of-concept that bridges academic deep learning research and practical application. Future work should focus on multi-forgery-type generalization, adversarial robustness, and integration of transformer-based architectures (e.g., ViT) for global context modeling.

GitHub Repository:
https://github.com/Deekshareddy10/Image_forgery_detection_using_DL/tree/main